Touchdown

The orbiter's photos show that the planned landing site is rougher than expected. Today you pick a safe site, land Viking 1, and talk home through the orbiter.

Marcus Birkenkrahe

COSC 1210 · CS I · Session 2 of 2 · Augustana University

Fall 2026 · Downey, *Think Python* 3e, ch. 6

Mission plan: two sessions, from orbit to the surface

Session 1: Phone home

`return`, `None`, pure functions, `assert`

Session 2 · today: Touchdown

Every path returns, Boolean functions, incremental development, recursion that returns a value, input validation

1. **Where to land:** rate the candidate sites
2. **Descent:** how much fuel do the engines need?
3. **On the ground:** count the pictures, talk home through the orbiter

Review · phone home: two quick-fire questions

1. What does a function return if it has no `return` statement?

→ None.

2. What does this print?

```
def to_kelvin(celsius):
    return celsius + 273.15

to_kelvin(-86)
print(to_kelvin(0))
```

→ Only `273.15`. The first call returns a value, but nobody stores or prints it.

Every path needs a return

→ Switch to editor · code-along 1 · TRY (2 min)

The landing team rates a site by its slope in degrees. Below 5 is safe, below 15 is risky – and anything steeper?

```
def site_rating(slope):
    if slope < 5:
        return "SAFE"
    elif slope < 15:
        return "RISKY"

print(site_rating(3))
print(site_rating(10))
print(site_rating(20))
```

1. Predict what `site_rating(20)` returns
2. Fix the function so that every path returns: anything steeper is `"NO-GO"`

A path without return falls off the end

```
SAFE
RISKY
None
```

For a slope of 20, no condition is True: the function runs off its end and returns None. The team gets no answer for exactly the sites that matter most.

```
def site_rating(slope):
    if slope < 5:
        return "SAFE"
        print("site logged")      # dead code: never runs
    elif slope < 15:
        return "RISKY"
    else:
        return "NO-GO"

print(site_rating(3))
print(site_rating(20))
```

```
SAFE
NO-GO
```

The print after return is **dead code**: return ends the function on the spot. Python does not warn you.

Question · predict first: which slopes return None?

```
def site_rating2(slope):
    if slope < 5:
        return "SAFE"
    if slope > 5 and slope < 15:
        return "RISKY"
    if slope > 15:
        return "NO-GO"
```

Answer: exactly 5 and exactly 15

```
print(site_rating2(5), site_rating2(15))
```

```
None None
```

The boundaries fall through all three conditions. An `if/elif/else` chain cannot have this gap – the `else` catches everything left over.

Boolean functions

→ Switch to editor · code-along 2 · TRY (2 min)

A function can return `True` or `False`. That gives a complicated test a name. A site is safe if its slope is below 10 degrees and fewer than 5 large rocks show up in the photo:

```
def is_safe(slope, rocks):
    if slope < 10 and rocks < 5:
        return True
    else:
        return False

print(is_safe(3, 2))
print(is_safe(3, 12))
```

```
True
False
```

TRY: Rewrite `is_safe` without `if`. Same input, same output.

Return the condition directly

The condition `slope < 10 and rocks < 5` already *is* `True` or `False`. Return it directly – no `if` needed:

```
def is_safe(slope, rocks):
    return slope < 10 and rocks < 5

if is_safe(3, 2):                # not: if is_safe(3, 2) == True:
    print("Site cleared for landing")
```

```
Site cleared for landing
```

Comparing with `== True` adds nothing: `is_safe(3, 2)` already is `True`.

Descent fuel: incremental development

→ **Switch to editor · code-along 3**

After the parachute, Viking 1's descent engines slow the lander down before touchdown. How much fuel does that take?

```
# time = (speed_start - speed_end) / deceleration  (seconds)
# fuel = burn_rate * time                          (kg)
# return fuel rounded to one decimal
```

Instead of writing it all at once and hoping, build it in small steps:

1. **Stub**: name, parameters, dummy `return`
2. **Add one piece** of real work
3. **Scaffold**: store it in a variable, `print` it
4. **Test** with inputs whose answer you know
5. **Repeat** 2–4, then **clean up**

Rule

If something breaks, the bug is in the last change.

Descent fuel: stub, then one piece

Stub:

```
def descent_fuel(speed_start, speed_end, deceleration, burn_rate):
    return 0.0

print(descent_fuel(60, 2.5, 5, 8))
```

```
0.0
```

Time + scaffold. From 60 m/s down to 2.5 m/s at 5 m/s per second should take 11.5 seconds:

```
def descent_fuel(speed_start, speed_end, deceleration, burn_rate):
    time = (speed_start - speed_end) / deceleration
    print("time:", time)
    return 0.0

descent_fuel(60, 2.5, 5, 8)
```

```
time: 11.5
```

Descent fuel: finish and clean up

→ **Switch to editor · TRY (5 min, in pairs)**

1. Add the fuel step and check it against 92 kg (8 kg/s for 11.5 s)
2. Clean up: remove the scaffolding and return the fuel rounded to one decimal
3. Second test: `descent_fuel(70, 2.5, 4, 8)` is `135.0`

```
def descent_fuel(speed_start, speed_end, deceleration, burn_rate):
    time = (speed_start - speed_end) / deceleration
    fuel = burn_rate * time
    return round(fuel, 1)

assert descent_fuel(60, 2.5, 5, 8) == 92.0
print(descent_fuel(70, 2.5, 4, 8))
```

```
135.0
```

Touchdown! Recursion that returns

→ Switch to editor · code-along 4 · TRY: predict `images(3)`

The lander's camera team plans its pictures: on sol 1 (a Mars day) it takes 1 image, on sol 2 it takes 2, and so on. How many images are there after n sols?

```
images(0) = 0
images(n) = n + images(n-1)
```

```
def images(n):
    if n == 0:
        return 0
    else:
        return n + images(n - 1)

print(images(3))
print(images(30))
```

```
6
465
```

countdown printed on its way down. images has to *add up* – so each call hands its result back.

Watch the values come back

→ Switch to editor · TRY (2 min)

Where does the 6 come from? Write down the eight lines this prints, in order, before you run it.

```
def images(n):
    print("images", n, "called")
    if n == 0:
        print("images 0 returns 0")
        return 0
    else:
        result = n + images(n - 1)
        print("images", n, "returns", result)
        return result

images(3)
```

```
images 3 called
images 2 called
images 1 called
images 0 called
images 0 returns 0
images 1 returns 1
images 2 returns 3
images 3 returns 6
```

Down to the base case, back up with the values

```
images(3)  = 3 + images(2)
           = 3 + 2 + images(1)
           = 3 + 2 + 1 + images(0)
           = 3 + 2 + 1 + 0 = 6
```

The calls go **down** to the base case like the countdown did. Then the return values come back **up**: each call adds its own `n` to the value it receives, and hands the sum to its caller.

The `print` lines were scaffolding: remove them when the function works.

The leap of faith

Following every call works for `images(3)`. For `images(30)` it becomes hopeless. Read recursion differently:

- When you call `round` or `to_kelvin`, you do not trace their code. You trust that they work
- Do the same with the recursive call. **Assume** `images(n - 1)` returns the right total. Then ask: is `n + images(n - 1)` right for `n`? Yes
- Check the base case separately. If both are right, the function is right

The leap of faith

Trusting a function you are still writing feels strange – that is why it is called the *leap of faith*.

Question · predict first: checksum for the downlink

Radio noise can flip a digit. The lander sends a **checksum** with every number: the sum of its digits. `1976 % 10` is 6, `1976 // 10` is 197.

```
def digit_sum(n):
    if n == 0:
        return 0
    else:
        return n % 10 + digit_sum(n // 10)

print(digit_sum(1976))
```

Leap of faith: `digit_sum(197)` returns 17. What does `digit_sum(1976)` return? Do not trace – use the 17.

```
23
```

Last digit plus the sum of the rest: $6 + 17 = 23$. Same shape as `images`: one piece of work plus one recursive call you trust. Each call drops a digit, so `n` reaches the base case 0.

Validate before you recurse

→ **Switch to editor · code-along 5**

What does `images(2.5)` do? Predict first.

2.5, 1.5, 0.5, −0.5, . . . it never equals 0:

```
>>> images(2.5)
RecursionError: maximum recursion depth exceeded
```

The built-in function `isinstance` checks the type of a value:

```
print(isinstance(3, int))
print(isinstance(2.5, int))
```

```
True
False
```

Guardrails before the recursion starts

A **guardrail** stops bad input early with a clear error: a `TypeError` for the wrong *type*, a `ValueError` for the right type with a wrong *value*.

```
def images(n):
    if not isinstance(n, int):
        raise TypeError("n must be a whole number of sols")
    elif n < 0:
        raise ValueError("n cannot be negative")
    elif n == 0:
        return 0
    else:
        return n + images(n - 1)

print(images(3))
print(images(2.5))
```

```
6
Traceback (...):
  ...
TypeError: n must be a
  whole number of sols
```

Once both checks pass, the recursion is certain to reach its base case.

Practice (if time): when does Earth hear touchdown?

→ **Switch to editor · incremental development, one step at a time**

The lander radios "touchdown" to Earth. Radio travels at about 300,000 km per second. How many minutes until Mission Control hears it?

```
# seconds = distance / 300000        (distance in km)
# minutes = seconds / 60
# return minutes rounded to one decimal
```

1. **Stub**: `signal_delay(distance)` returns `0.0`
2. **Seconds + scaffold**: `signal_delay(300000)` prints `1.0` seconds
3. **Minutes + scaffold**: 18,000,000 km take 60 seconds – `1.0` minute
4. **Clean up**: return the rounded minutes, `assert signal_delay(18000000) == 1.0`
5. July 20, 1976: Mars was about 340,000,000 km away. `print(signal_delay(340000000))` prints `18.9`

Checkout

1. Why is `return slope < 10 and rocks < 5` enough – no `if` needed?

→ The condition already evaluates to `True` or `False`.

2. Why does `images(2.5)` not stop without validation?

→ The argument goes 1.5, 0.5, −0.5, ... and never equals 0, so the base case is never reached.

Summary

- Every path through a function needs a `return`. A path without one falls off the end and returns `None` – check the boundaries
- Code after a `return` in the same branch is **dead code**
- A **Boolean function** returns `True` or `False`. Return the condition directly, and use the function in `if` without `== True`
- **Incremental development**: stub, add one piece, scaffold with `print`, test, repeat, clean up. A bug is always in the last change
- A recursive function can **return** a value: the calls go down to the base case, the return values travel back up
- The **leap of faith**: assume the recursive call is right, check the current step and the base case
- A **guardrail** (`isinstance`, `raise TypeError` / `ValueError`) stops bad input early and makes sure a recursion reaches its base case

Glossary: terms from session 2

#	Term	Meaning
1	dead code	Code that never runs, e.g. after `return`
2	Boolean function	Returns `True` or `False`
3	incremental development	Add and test a little code at a time
4	stub	First version of a function, dummy `return`
5	scaffolding	Temporary code, e.g. `print`, for testing
6	sol	A day on Mars, about 24 h 40 min
7	leap of faith	Trusting a (recursive) call to be right
8	guardrail	Stops bad input early with a clear error
9	`isinstance`	Checks whether a value has a given type
10	`TypeError` / `ValueError`	Error for a wrong type / a wrong value
11	greatest common divisor	Largest number that divides both exactly

Next: loops

Recursion repeats by calling itself. Next chapter, Python gets a second way to repeat:

- Loops that walk through strings and files

Homework (details in the script)

1. Guardrail for `descent_fuel`: `ValueError` if the deceleration is 0 or less
2. Guardrail for `signal_delay`: `ValueError` if the distance is 0 or less
3. Relay windows: `gcd` with Euclid's algorithm, then `lcm`